Duvar

Point-Based Neural Mapping Framework

License	GNU General Public License v3.0
Architecture	PBNM-Flow (Point-Based Neural Mapping)
Reference model	TinyLlama-1.1B-Chat-v1.0
Runtime	Google Colab T4 GPU (16 GB VRAM)

1. Overview

Duvar (Turkish: *Wall*) is an experimental neural architecture that replaces standard sequential layer execution with a pointer-based directed graph. Instead of PyTorch iterating over a fixed `self.layers` list, each transformer block carries a `target_ptr` — a declared pointer to its successor. A custom forward pass reads those pointers at runtime and traverses the computation graph accordingly.

The result: the execution path of a given inference is an explicit, inspectable sequence of block identifiers — not a side effect buried in module iteration, not an approximation reconstructed by gradient attribution. The route is a first-class artifact of the architecture.

2. Architecture

2.1 Components

Component	Role
PointEntry	Fundamental data unit: weight pointer + target layer ID
DuvarLinear	Drop-in nn.Linear replacement with per-row INT8 quantization and routing pointer
DuvarGraph	Directed pointer graph over all linear layers (compression and metadata)
DuvarBlockGraph	High-level routing graph over transformer decoder blocks (runtime control)
duvar_forward()	Custom forward: traverses blocks via DuvarBlockGraph pointers
duvar_generate()	Custom generation loop calling duvar_forward at every token step
Triton Kernels	Fused FP16 ops: SiLU, GeLU, RMSNorm, INT8 dequantization

2.2 The Brick Abstraction

A `PointEntry` is the atom of the Duvar system — a brick: a self-contained unit holding a reference to its weight tensor and a pointer to the next brick (`target_layer`). A `DuvarLinear` wraps a standard linear operation and adds per-row INT8 quantization and a `next_layer_ptr` field. A `DuvarGraph` records the full directed pointer table over every linear layer and is serialized to

`duvar_metadata.json`.

2.3 Block-Level Routing: DuvarBlockGraph

`DuvarGraph` operates at the linear-layer level and serves compression metadata. `DuvarBlockGraph` operates at the transformer-block level and drives inference. It maps block indices (integers) to their successors (integers or the sentinel string `'OUTPUT'`). The default routing is sequential; any pointer can be patched at runtime without touching weight tensors.

```
DuvarBlockGraph
class DuvarBlockGraph:
    def __init__(self, num_layers: int):
        # Default: sequential  0 -> 1 -> 2 -> ... -> N-1 -> 'OUTPUT'
        self.routing = {i: (i+1 if i+1 < num_layers else 'OUTPUT')
                        for i in range(num_layers)}
    def successor(self, idx: int):
        return self.routing.get(idx, 'OUTPUT')
    def set_route(self, patches: dict):
        self.routing.update(patches)   # e.g. {9: 16} skips blocks 10-15
    def active_path(self) -> list:
        path, current = [], 0
        while current != 'OUTPUT':
            path.append(current)
            current = self.successor(current)
        return path
```

3. PBNM-Flow Forward Pass

`duvar_forward()` replaces HuggingFace's `model.generate()` and PyTorch's implicit module iteration. Traversal is explicit: the function reads the next block index from `DuvarBlockGraph.successor()` at every step. An optional `route_log` argument records the visited block indices — producing a concrete execution trace with zero additional overhead.

```
duvar_forward — core routing loop
def duvar_forward(model, input_ids, block_graph, route_log=None):
    hidden_states = model.model.embed_tokens(input_ids)
    position_ids  = torch.arange(seq_len, device=device).unsqueeze(0)
    causal_mask   = _build_causal_mask(seq_len, device)
    current = 0
    while current != 'OUTPUT':
        layer         = model.model.layers[current]
        hidden_states = layer(hidden_states, causal_mask, position_ids)[0]
        if route_log is not None:
            route_log.append(current)   # XAI: log every block visited
        current = block_graph.successor(current)
    hidden_states = model.model.norm(hidden_states)
    return model.lm_head(hidden_states.float())
```

`duvar_generate()` wraps this into a full generation loop with top-p sampling, deterministic seeding, and a route log that records the execution path of the final token step. It does not rely on HuggingFace's generation infrastructure — the routing logic is the generation infrastructure.

4. Routing Modes

Because traversal order is declared in `DuvarBlockGraph`, the execution path is a runtime parameter, not a code constant. Two routing modes are demonstrated in the notebook.

Sequential (default)

Pointer table: 0 → 1 → 2 → ... → 21 → OUTPUT. Identical computation to standard transformer forward pass. Used as a parity check against the FP16 baseline.

Sparse / skip routing

```
Skip-routing example
sparse_graph = DuvarBlockGraph(22)
sparse_graph.set_route({9: 16})   # jump from block 9 to block 16
# active_path() -> [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 16, 17, 18, 19, 20, 21]
```

Blocks 10 through 15 are bypassed entirely. No retraining. No masking. No weight surgery. The change is one dictionary entry. The route log output is concrete:

```
Route log — inspectable execution trace
Route (last token): [0,1,2,3,4,5,6,7,8,9,16,17,18,19,20,21]
```

5. Quantization Scheme (INT8, not FP8)

Weights are stored as `uint8` with per-row scale and min parameters. FP8 (e4m3/e5m2) is a floating-point format used in hardware like the H100 and is architecturally distinct. For each row *r* of a weight matrix:

```
Per-row INT8 quantization
scale[r] = (max(w[r]) - min(w[r])) / 255
index[r] = round((w[r] - min(w[r])) / scale[r])   in [0, 255]
# Reconstruction (Triton kernel):
w[r] = index[r] * scale[r] + min(w[r])
```

Weights are dequantized to FP16 before the matmul. Hardware matmul units are faster in FP16; the INT8 format serves memory bandwidth, not arithmetic throughput. The Triton `duvar_dequant` kernel fuses the dequantization into a single pass to avoid a separate read-write over the weight buffer.

6. Triton Kernels

Five fused kernels replace the standard PyTorch operations in the hot path. Each falls back to a pure-PyTorch implementation with identical mathematics if Triton is unavailable.

Kernel	Operation	Benefit
duvar_dequant	Per-row INT8 → FP16	Fused to avoid a separate read-write pass over weights
duvar_silu	x · σ(x)	Avoids materializing full FP32 intermediate tensor
duvar_gelu	0.5x(1 + tanh(...))	Numerically stable tanh via sigmoid identity

Kernel	Operation	Benefit
duvar_relu	max(x, 0)	Baseline activation for ablation studies
duvar_rmsnorm	$x / \sqrt{(\text{mean}(x^2)+\varepsilon)} \cdot w$	Row-parallel normalization

7. Performance

7.1 Size

Linear weight compression: approximately 2x (FP16 → INT8 per-row). Embeddings, norms, and the tokenizer are not quantized; total file compression is slightly below 2x due to these unquantized components.

7.2 Speed

On memory-bandwidth-constrained hardware (T4), the bandwidth savings from loading INT8 instead of FP16 weights outweigh the per-layer Triton dequantization cost. Measured: approximately 12s FP16 baseline versus 11s Duvar (INT8 + Triton dequant) on 200-token generation. The `duvar_generate()` loop does not use a KV cache (straightforward to add; omitted to keep routing logic transparent). Skip routing demonstrates measurable additional latency reduction proportional to the number of bypassed blocks.

7.3 Output Quality

Semantic similarity between FP16 baseline and INT8 Duvar outputs is measured via Jaccard similarity over word sets. Per-row quantization introduces small reconstruction errors; maximum dequant error is verified less than 0.001 in FP16 precision bounds (Cell 13). Jaccard similarity is a word-overlap proxy; perplexity on a held-out corpus is the correct formal evaluation.

8. Explainability (XAI) Properties

Duvar produces explainability as a structural consequence of pointer-driven traversal.

Property	How Duvar Enables It
Route Traceability	duvar_forward() logs every block index visited. The execution path is a concrete list of integers, not a g
Modular Ablation	Changing a route pointer redirects computation at the block level without modifying any weight tensor.
Spatial Activation Mapping	Blocks visited frequently by a class of inputs are structurally distinguishable from bypassed blocks. Hea
Declared Sparse Execution	active_path() returns the exact set of blocks that will be computed before inference begins. The work is

Existing XAI tools (LIME, SHAP) estimate attribution from outside the model. Duvar exposes the execution path from inside.

9. Comparison with Prior Work

Prior Art	Similarity	Key Difference
PathNet (DeepMind)	Path-based traversal through a supernetwork	PathNet uses evolutionary algorithms to find paths during training; Duvar expose
Switch Transformers / MoE	Dynamic dispatch over sub-networks	MoE operates on a fixed backbone; Duvar breaks the backbone into a hot-swapp
LangGraph	Nodes and edges for flow control	LangGraph routes at the agent/application level; Duvar routes at the tensor/layer
Deep Equilibrium Models	Non-sequential computation	DEQ uses fixed-point iteration; Duvar's bricks are independent and the path is ex

10. Theoretical Claims

Topology over arithmetic. Replacing sequential block iteration with pointer-driven traversal enables sparse execution — only the blocks relevant to a given input need to activate. Demonstrated with `set_route({9: 16})` producing a 6-block shorter path with measurable latency reduction.

Bandwidth over latency. INT8 storage reduces VRAM-to-SM data movement. On bandwidth-bound hardware (T4), this is the dominant cost. Triton fused kernels minimize dequantization overhead.

Composable routing. Any `DuvarBlockGraph` is serializable and reversible. A route used for one input can be stored, replayed, and compared against another route — making inference topology an artifact of the same kind as a weight checkpoint.

11. Honest Limitations

The sparse execution gains from topology-driven routing become significant at scale — 70B+ parameter models and distributed inference across many devices. TinyLlama on a T4 demonstrates the mechanism; it does not produce the magnitude of gain the architecture is designed for.

`duvar_generate()` does not implement a KV cache. This is a deliberate clarity trade-off for the demo; pointer routing and KV cache are orthogonal and combining them is straightforward.

INT8 quantization introduces small numerical errors. Jaccard similarity is a word-overlap proxy, not a formal quality evaluation. A perplexity comparison on a held-out corpus is the correct next step.

12. Usage

12.1 Requirements

```
requirements
transformers==4.40.0
accelerate
sentencepiece
protobuf
huggingface_hub
triton   # optional but recommended
```

12.2 Cell Execution Order (Google Colab T4)

Cell	Action
1	Environment setup (~60s)
2	Imports and GPU verification
3	Download TinyLlama-1.1B (~2.2 GB, cached after first run)
4	Compile Triton kernels
5	Load Duvar architecture classes (DuvarLinear, DuvarGraph, DuvarBlockGraph)
6	FP16 baseline inference
7	Convert model to Duvar (INT8 + pointer graphs)
8	Save converted model
9	PBNM-Flow runtime: duvar_forward + duvar_generate
10	Sequential route inference (baseline parity check)
11	Sparse route inference (skip demo + route log)
12	System report
13	Kernel numerical accuracy verification
14	Per-token latency benchmark

12.3 Output Files

File	Contents
duvar_weights.pt	Full model state dict with INT8 linear weights
duvar_metadata.json	Linear-layer pointer graph, compression stats, architecture info
tokenizer_config.json + supporting files	Saved tokenizer
